# Obstacle-Aware Quadrotor Control via Nonlinear MPC and Artificial Potential Fields

Gizem Doğa Filiz, İsmail Cem Yılmaz, Ivano Bazzo

***Abstract*— This paper presents a quaternion-based Nonlinear Model Predictive Control (NMPC) framework for quadrotor obstacle avoidance and aggressive maneuvers, and studies *where* obstacle avoidance should be performed: as formal collision-avoidance constraints *inside* the NMPC optimal control problem, or by a separate Artificial Potential Field (APF) planner that generates references the NMPC merely tracks. The controller uses a 13-dimensional rigid-body state with quaternion attitude, avoiding Euler-angle singularities, and is solved online with an acados SQP-RTI solver at a 1 s horizon and 50 ms sampling rate; obstacles are detected online from a 2-D lidar via DBSCAN clustering. Evaluated in Gazebo on a five-obstacle in-line slalom at a matched ≈0.9 m clearance, the in-MPC keep-out constraint navigates reliably up to 1.5 m/s while the APF-reference plus NMPC-tracker reaches 2.2 m/s; both fail at higher speed through the same attitude-runaway instability, and the APF advantage is shown to stem from its wider look-ahead rather than from the architecture itself. A backflip maneuver is additionally executed with a feedforward Lupashin torque profile and NMPC-based recovery, which rejects the post-flip excursion and settles within 0.1 m of the hover point. The average solver time is 8.6 ms, well within the 50 ms control budget.**

## I. Introduction

Autonomous quadrotor navigation in cluttered environments requires real-time trajectory planning and precise low-level control. Classical approaches decouple these tasks: a high-level planner generates a geometric path, while a separate controller tracks it. Artificial Potential Fields (APF) [1] provide a computationally lightweight mechanism for reactive path planning, yet they lack inherent guarantees on dynamic feasibility. Nonlinear Model Predictive Control (NMPC) [3], in contrast, explicitly enforces vehicle dynamics and actuation limits over a receding horizon, but requires an accurate reference trajectory. Within NMPC, collision avoidance can be embedded directly as soft or hard constraints or as control barrier functions [6]; recent agile-flight work has further compared NMPC against differential-flatness tracking [7].

These two mechanisms can both prevent collisions, but they place the avoidance logic in different parts of the stack. This study therefore poses a concrete question: *should obstacle avoidance be enforced as formal collision-avoidance constraints inside the NMPC, or delegated to an APF planner whose collision-free references the NMPC only tracks?* Both are implemented on the same quaternion NMPC and the same lidar perception, and compared on an in-line slalom course at a deliberately matched obstacle clearance, so that only the architecture differs. The main contributions are:

- A quaternion-based NMPC formulation, solved with acados SQP-RTI, with both a landing-cone soft constraint and online soft *keep-out* collision-avoidance constraints embedded in the optimal control problem.
- A reactive APF horizon builder that feeds per-step references to the NMPC, enabling online avoidance without pre-planned waypoints, with lidar-based obstacle detection via DBSCAN clustering providing the obstacles to both modes.
- A controlled comparison of in-MPC keep-out avoidance against APF-reference tracking at matched clearance, identifying a shared attitude-runaway speed limit and showing that the APF speed advantage stems from its wider look-ahead, not the architecture.
- A backflip maneuver combining a feedforward Lupashin torque profile [5] with NMPC-based post-flip recovery.

The remainder of the paper is organised as follows. Section II presents the methodology, covering the dynamic model, NMPC formulation, APF planner, lidar perception, and backflip maneuver. The experimental setup is described in Section III. Results are presented and discussed in Section IV. Finally, Section V concludes the paper.

## II. Methodology

### A. Quadrotor Dynamic Model

*1) State and input definitions:* The quadrotor is modelled as a rigid body with mass $m$ and diagonal inertia $\mathbf{J} = \mathrm{diag}(I_{xx}, I_{yy}, I_{zz})$ (numerical values in Table I). The system state is

$$\mathbf{x} = \begin{bmatrix}\mathbf{p}^\top & \mathbf{v}^\top & \mathbf{q}^\top & \boldsymbol{\omega}^\top\end{bmatrix}^\top \in \mathbb{R}^{13}, \tag{1}$$

where $\mathbf{p} \in \mathbb{R}^3$ is the world-frame position, $\mathbf{v} \in \mathbb{R}^3$ the world-frame linear velocity, $\mathbf{q} = [q_w, q_x, q_y, q_z]^\top \in \mathbb{S}^3$ the unit quaternion, and $\boldsymbol{\omega} = [p, q, r]^\top \in \mathbb{R}^3$ the body-frame angular velocity. The control input is the generalised wrench

$$\mathbf{u} = \begin{bmatrix} f_{\mathrm{tot}} & \tau_x & \tau_y & \tau_z \end{bmatrix}^\top \in \mathbb{R}^4. \tag{2}$$

*2) Equations of motion:* Newton's second law in the world frame gives $\dot{\mathbf{p}} = \mathbf{v}$ and

$$\dot{\mathbf{v}} = \tfrac{f_{\mathrm{tot}}}{m}\,\mathbf{R}(\mathbf{q})\,\mathbf{e}_3 - g\,\mathbf{e}_3, \tag{3}$$

where $\mathbf{R}(\mathbf{q}) \in SO(3)$ and $g = 9.81\,\mathrm{m/s^2}$. Expanding the quaternion–rotation product:

$$\dot{v}_x = \tfrac{2(q_w q_y + q_x q_z)}{m} f_{\mathrm{tot}}, \tag{4}$$

$$\dot{v}_y = \tfrac{2(q_y q_z - q_w q_x)}{m} f_{\mathrm{tot}}, \tag{5}$$

$$\dot{v}_z = \tfrac{1-2q_x^2-2q_y^2}{m} f_{\mathrm{tot}} - g. \tag{6}$$

Quaternion kinematics are governed by

$$\dot{\mathbf{q}} = \tfrac{1}{2}\,\boldsymbol{\Xi}(\mathbf{q})\,\boldsymbol{\omega}, \qquad \boldsymbol{\Xi}(\mathbf{q}) = \begin{bmatrix} -q_x & -q_y & -q_z \\ q_w & -q_z & q_y \\ q_z & q_w & -q_x \\ -q_y & q_x & q_w \end{bmatrix}. \quad (7)$$

The rotational dynamics follow Euler's rigid-body equation:

$$\dot{p} = \big[\tau_x - (I_{zz} - I_{yy})qr\big]/I_{xx}, \quad (8)$$

$$\dot{q} = \big[\tau_y - (I_{xx} - I_{zz})pr\big]/I_{yy}, \quad (9)$$

$$\dot{r} = \big[\tau_z - (I_{yy} - I_{xx})pq\big]/I_{zz}. \quad (10)$$

The full model is written compactly as $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$.

*3) Motor allocation:* Individual squared rotor speeds $\boldsymbol{\Omega}^2 \in \mathbb{R}^4$ are obtained by inverting the "+"-configuration allocation matrix $\mathbf{u} = \mathbf{B}\boldsymbol{\Omega}^2$, where

$$\mathbf{B} = \begin{bmatrix} k_f & k_f & k_f & k_f \\ 0 & \ell k_f & 0 & -\ell k_f \\ -\ell k_f & 0 & \ell k_f & 0 \\ k_m & -k_m & k_m & -k_m \end{bmatrix}, \quad (11)$$

with thrust and moment coefficients $k_f$, $k_m$ and arm length $\ell$ (Table I).

### B. Nonlinear Model Predictive Control

*1) Optimal control problem:* At each sampling instant $t_k$ the NMPC solves the finite-horizon OCP

$$\min_{\mathbf{x}_{0:N}, \mathbf{u}_{0:N-1}} \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + \ell_N(\mathbf{x}_N) \quad (12)$$

subject to

$$\mathbf{x}_{k+1} = F(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, N-1, \quad (13)$$

$$\mathbf{x}_0 = \hat{\mathbf{x}}(t_k), \quad (14)$$

$$\mathbf{u}_k \in \mathcal{U}, \quad k = 0, \dots, N-1, \quad (15)$$

$$h(\mathbf{x}_k) \geq 0, \quad k = 0, \dots, N, \quad (16)$$

where $F(\cdot)$ is the discrete-time model obtained by integrating $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ with a four-stage explicit Runge-Kutta (ERK4) scheme, and $\hat{\mathbf{x}}(t_k)$ is the current state estimate. The horizon parameters $N$ and $T_s$ are given in Table I.

*2) Cost function:* A NONLINEAR_LS cost structure is adopted with output map $\mathbf{h}(\mathbf{x}, \mathbf{u}) = [\mathbf{x}^\top, \mathbf{u}^\top]^\top \in \mathbb{R}^{17}$:

$$\ell(\mathbf{x}_k, \mathbf{u}_k) = \big\|\mathbf{h}(\mathbf{x}_k, \mathbf{u}_k) - \mathbf{y}_k^{\mathrm{ref}}\big\|^2_{\mathbf{W}}, \quad (17)$$

where $\mathbf{y}_k^{\mathrm{ref}} = [\mathbf{x}_k^{\mathrm{ref}\top}, \mathbf{u}_{\mathrm{hov}}^\top]^\top$ with $\mathbf{u}_{\mathrm{hov}} = [mg, 0, 0, 0]^\top$. The weight matrix $\mathbf{W} = \mathrm{diag}(\mathbf{Q}, \mathbf{R})$ is

$$\mathbf{Q} = \mathrm{diag}(\underbrace{5, 5, 5}_{\mathbf{p}}, \underbrace{3, 3, 3}_{\mathbf{v}}, \underbrace{1.5, 1.5, 1.5, 1.5}_{\mathbf{q}}, \underbrace{25, 25, 6}_{\boldsymbol{\omega}}), \quad (18)$$

$$\mathbf{R} = \mathrm{diag}(0.01,\ 0.10,\ 0.10,\ 0.20). \quad (19)$$

The reduced yaw-rate weight $Q_r = 6 \ll Q_{p,q} = 25$ reflects the lower yaw authority relative to roll/pitch. The terminal cost is

$$\ell_N(\mathbf{x}_N) = \big\|\mathbf{x}_N - \mathbf{x}_N^{\mathrm{ref}}\big\|^2_{\mathbf{W}_N}, \quad \mathbf{W}_N = \mathrm{diag}(P_s\mathbf{Q}_{\mathbf{p},\mathbf{v},\mathbf{q}},\, \mathbf{0}_3), \quad (20)$$

with $P_s = 5$ and zero terminal penalty on angular rates.

*3) Constraints:* The feasible set $\mathcal{U}$ enforces actuator limits (Table I). The tight yaw-torque limit reflects the physical limit $f_{\mathrm{hov}}k_m/k_f \approx 0.19\,\mathrm{Nm}$ imposed by the motor mixer; the conservative bound eliminates yaw-saturation oscillations observed during validation.

To prevent hard landings a soft state constraint is imposed at every stage:

$$h_{\mathrm{land}}(\mathbf{x}_k) = v_z + \alpha p_z \geq 0, \quad \alpha = 2.0\ \mathrm{s}^{-1}. \quad (21)$$

At $p_z = 0.3\,\mathrm{m}$ the maximum permissible descent rate is $0.6\,\mathrm{m/s}$. The constraint is softened via a lower-bound slack $s_k \geq 0$, adding $W_{\mathrm{land}}s_k^2$ to the stage cost with $W_{\mathrm{land}} = 500$.

*4) Obstacle keep-out constraints:* When obstacle avoidance is enforced *inside* the NMPC, a soft keep-out constraint is imposed at every stage for each obstacle:

$$h_{\mathrm{obs},i}(\mathbf{x}_k) = \|\mathbf{p}_k - \mathbf{p}_{o,i}\|^2 - (r_i + R_d)^2 \geq 0, \quad (22)$$

where $\mathbf{p}_{o,i}$ and $r_i$ are the centre and radius of obstacle $i$ and $R_d$ is the drone radius (Table I). The obstacle centres and radii are online parameters, refreshed each control step from the lidar. Each constraint is softened with a lower-bound slack penalised by $W_{\mathrm{obs}} = 10^4$, so a feasible control is always returned even under transient perception errors. The *same* NMPC acts as a pure tracker when no obstacles are supplied (APF-reference mode); activating (22) makes the controller itself responsible for clearance (in-MPC mode). Because the keep-out acts on the NMPC's dynamically-feasible predicted trajectory, it constrains the path the vehicle actually flies; an APF reference, by contrast, is only kinematically collision-free and may be tracked with error, so the executed path can pass closer than the reference intends.

*5) SQP real-time iteration solver:* The OCP (12) is solved with the *Real-Time Iteration* (RTI) variant of Sequential Quadratic Programming as implemented in acados [3]. RTI performs a single SQP iteration per sampling interval: the nonlinear dynamics are linearised around the current iterate, the resulting convex QP is solved with HPIPM [4] using partial condensing, and the solution is immediately applied to the plant. The Hessian is approximated via Gauss–Newton, $\nabla^2\mathcal{L} \approx \mathbf{J}_h^\top\mathbf{W}\mathbf{J}_h$, guaranteeing positive semi-definiteness. The solver configuration and all key parameters are summarised in Table I.

### C. APF-Based Obstacle Avoidance

*1) Potential field formulation:* Following [1], the workspace is modelled as a superposition of an attractive potential pulling toward the goal and repulsive potentials centred on obstacles. The corresponding force at position $\mathbf{p} \in \mathbb{R}^2$ is $\mathbf{F} = \mathbf{F}_{\mathrm{att}} + \mathbf{F}_{\mathrm{rep}}$, where

$$\mathbf{F}_{\mathrm{att}} = k_{\mathrm{att}}(\mathbf{p}_g - \mathbf{p}), \quad \|\mathbf{F}_{\mathrm{att}}\| \leq k_{\mathrm{att}}, \quad (23)$$

and, for each obstacle $i$ with centre $\mathbf{p}_{o,i}$, radius $r_i$, and influence distance $d_0$,

$$\mathbf{F}_{\mathrm{rep},i} = k_{\mathrm{rep}}\left(\frac{1}{\rho_i} - \frac{1}{d_0}\right)\frac{1}{\rho_i^2}\left(\hat{\mathbf{r}}_i + \tfrac{1}{2}\hat{\boldsymbol{\tau}}_i\right), \quad \rho_i < d_0, \quad (24)$$

with $\rho_i = \|\mathbf{p} - \mathbf{p}_{o,i}\| - r_i - R_\mathrm{d}$ the effective margin, $\hat{\mathbf{r}}_i$ the unit radial direction, and $\hat{\boldsymbol{\tau}}_i$ a signed tangent component. The tangent sign is precomputed from the cross product of the start-to-goal direction and the obstacle offset vector, ensuring consistent obstacle avoidance direction. The APF parameters $k_\mathrm{att}$, $k_\mathrm{rep}$, $d_0$, and $R_\mathrm{d}$ are listed in Table I.

*2) Reactive horizon builder:* At each control step the function $\mathbf{F}(\mathbf{p})$ is evaluated at the current position and integrated forward for $N+1$ steps of duration $T_s$ to build the NMPC reference horizon:

$$\mathbf{p}_{k+1} = \mathbf{p}_k + T_s \cdot v_\mathrm{max} \frac{\mathbf{F}(\mathbf{p}_k)}{\|\mathbf{F}(\mathbf{p}_k)\|} \cdot \min\left(1,\ \frac{d_k}{2}\right), \quad (25)$$

where $d_k$ is the distance to goal and $v_\mathrm{max}$ is the maximum speed (Table I). No pre-planned path is required; the system responds online to changes in obstacle configuration.

This builder is used in two regimes: a wide influence distance $d_0$ (smooth, early avoidance) and a tight one matched to the in-MPC clearance. The in-MPC mode instead uses a goal-only reference and delegates avoidance entirely to the keep-out constraint (22).

### D. Lidar-Based Perception

The obstacle avoidance pipeline is driven by a 2-D lidar sensor mounted on the quadrotor. At each control step, the lidar point cloud is segmented using DBSCAN clustering with parameters $\varepsilon$ and $n_\mathrm{min}$ given in Table I. Each cluster is fitted with a bounding circle to estimate the obstacle centre $\mathbf{p}_{o,i}$ and radius $r_i$, which are supplied to the active avoidance module: the NMPC keep-out constraint (22) in in-MPC mode, or the APF force computation in APF-reference mode. The perception module runs synchronously with the NMPC at 20 Hz, adding negligible latency ($< 1$ ms).

### E. Backflip Maneuver

A full 360° backflip is executed using the five-phase bang-coast-bang strategy of Lupashin *et al.* [5]. Driving flips open-loop with a feedforward torque profile is the established approach: in [5] the bang-bang profile is refined across repeated trials by iterative learning, and learning-based acrobatic controllers such as Deep Drone Acrobatics [8] distil an optimal-control expert into a neural-network policy rather than running an MPC through the maneuver online. The reason is structural: the full rotation takes longer than the $T = 1.0$ s prediction horizon, so a receding-horizon controller cannot plan through it. We therefore adopt the same split: the flip rotation is driven open-loop by the feedforward profile, while the NMPC handles the preparation (popup) and, most importantly, the post-flip recovery, where closed-loop control matters most.

The five phases are:

1) **Popup**: NMPC active; thrust increased to $f_\mathrm{pop}$ (Table I) to gain altitude margin.
2) **Accel**: NMPC off; maximum pitch torque applied to initiate rapid rotation.
3) **Coast**: near-zero thrust; angular momentum carries the rotation at peak pitch rate.
4) **Decel**: opposing pitch torque arrests the rotation.
5) **Recovery**: a brief open-loop rate-kill first damps the residual body rates, after which the NMPC is reactivated and stabilises the quadrotor from the large attitude and position error back to hover.

The recovery phase is the most demanding: after flip exit the vehicle has drifted laterally and retains residual angular rates. The rate-kill brings the rates low enough for the NMPC to converge, and the NMPC then recovers the attitude, arrests the lateral motion and returns the vehicle to the hover point.

## III. Experimental Validation

### A. Simulation Environment

All experiments were conducted in Gazebo using the tk3lab Docker environment. The simulated quadrotor is equipped with a 2-D lidar sensor for obstacle detection. The NMPC runs at 20 Hz on a standard desktop CPU; the perception module (DBSCAN clustering) runs synchronously at the same rate. All system and controller parameters are listed in Table I.

TABLE I: System and Controller Parameters

| Category | Parameter | Value |
|---|---|---|
| Vehicle | Mass $m$ | 1.280 kg |
| | Inertia $(I_{xx}, I_{yy}, I_{zz})$ | $(22.9, 22.9, 22.1) \times 10^{-3}$ kg m$^2$ |
| | Arm length $\ell$ | 0.23 m |
| | Thrust coeff. $k_f$ | $6.5 \times 10^{-4}$ N/(rad/s)$^2$ |
| | Moment coeff. $k_m$ | $10^{-5}$ N m/(rad/s)$^2$ |
| MPC | Horizon $N$ / $T_s$ | 20 / 0.05 s |
| | NLP solver | SQP-RTI (acados) |
| | QP solver | HPIPM (partial cond.) |
| | Integrator | ERK4 |
| | Thrust bounds | $[0.4,\ 2.5]\, mg$ |
| | Torque bounds | $\lvert\tau_{x,y}\rvert \le 0.25$, $\lvert\tau_z\rvert \le 0.06$ Nm |
| | Keep-out $R_d$/$W_\mathrm{obs}$/$n_\mathrm{obs}$ | 0.35 m / $10^4$ / 5 |
| APF | $k_\mathrm{att}$ / $k_\mathrm{rep}$ | 1.0 / 0.8 |
| | $d_0$ (wide / tight) | 2.5 / 0.3 m |
| | $R_d$ (wide / tight) | 0.50 / 0.20 m |
| | Cruise $v_\mathrm{max}$ | 1.5–2.8 m/s (swept) |
| Perception | DBSCAN $\varepsilon$ | 0.25 m |
| | Min. cluster $n_\mathrm{min}$ | 3 points |
| Backflip | Popup thrust $f_\mathrm{pop}$ | $2mg$ |
| | Flip torque $\tau_\mathrm{flip}$ | 1.1 Nm |

### B. Experimental Procedure

Two scenarios are evaluated:

- **In-line slalom**: five cylinders of radius 0.4 m placed on the straight start–goal line, navigated in three configurations (pure-MPC, APF+MPC tight, APF+MPC wide). For each, the commanded speed is swept and the maximum reliable speed is recorded over repeated runs; obstacles are detected online via lidar and DBSCAN.
- **Backflip**: executed from a 10 m hover altitude using the five-phase feedforward profile described in Section II-E, with NMPC-based recovery.

## IV. Results and Discussion

### A. Obstacle Avoidance: In-MPC Constraint vs. APF Reference

To isolate the effect of *where* avoidance is performed, all runs use the same in-line slalom: five cylinders of radius 0.4 m placed on the straight start–goal line at $(3.5, 0.3)$, $(6.5, -0.3)$, $(9.5, 0.3)$, $(12.5, -0.3)$ and $(15.5, 0.3)$ m along an 18 m corridor. Because the obstacles sit on the line, a goal-directed reference points straight through them, so any clearance must be produced by the avoidance mechanism itself. Obstacles are detected online by lidar and DBSCAN.

*1) Pure-MPC: avoidance inside the controller:* The *pure-MPC* configuration uses a goal-only reference together with the keep-out constraint (22), so avoidance lives entirely in the controller. Fig. 1 details a representative run at a minimum clearance of $\approx 0.9$ m: since a goal-only reference points straight through the obstacles, the lateral motion is produced entirely by the keep-out constraint, which holds the clearance to every obstacle above $R_d$ while thrust and torque stay within their bounds.

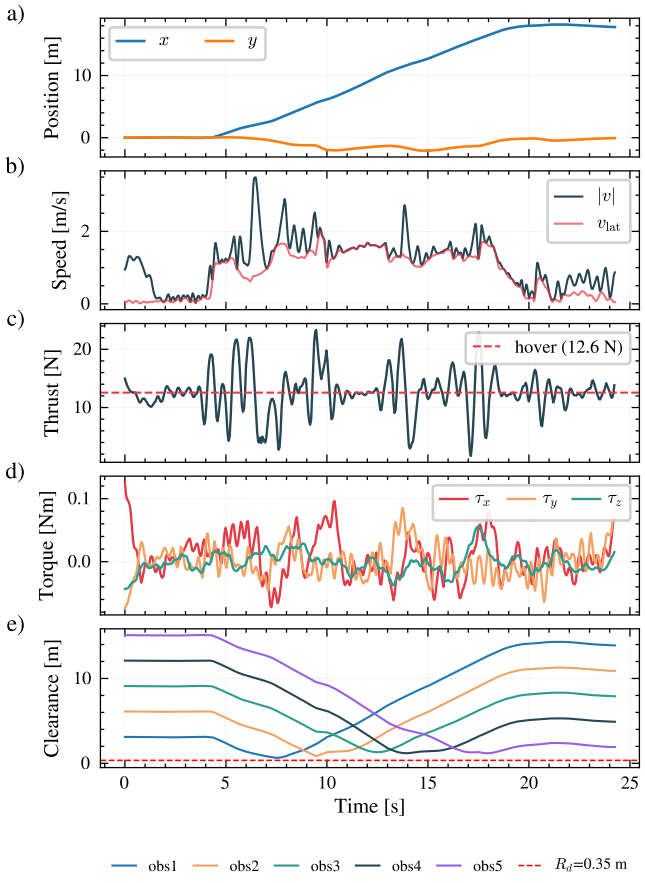


Fig. 1: Pure-MPC slalom flight (avoidance in the keep-out constraint). (a) Position. (b) Total and lateral speed. (c) Total thrust (hover $mg = 12.6$ N, dashed). (d) Body-frame torques $\tau_x$, $\tau_y$, $\tau_z$. (e) Per-obstacle clearance ($R_d = 0.35$ m, dashed).

*2) Comparison with APF-reference tracking:* Pure-MPC is compared against two configurations that instead delegate avoidance to an APF planner tracked by the same NMPC:

- **APF+MPC (tight)**: the APF influence and radius reduced ($d_0 = 0.3$ m, $R_d = 0.20$ m) to match the $\approx 0.9$ m clearance of pure-MPC.
- **APF+MPC (wide)**: the APF at its design point ($d_0 = 2.5$ m, $R_d = 0.50$ m), keeping a much larger $\approx 2.5$ m clearance.

Fig. 2(a) shows the resulting paths: the matched-clearance runs (pure-MPC, APF+MPC tight) thread within $\approx 1$ m of the obstacles, whereas the wide planner keeps much more clearance. The path *shape* also differs by where avoidance is decided. The APF feeds the resultant potential-field vector at the current position as a velocity reference, so the reference already curves away from an obstacle while it is still far, and the tracked path bends gradually and stays smooth. Pure-MPC instead carries a goal-only reference that points straight through the obstacle; the lateral motion appears only when the predicted trajectory reaches the keep-out boundary (22), producing a later, sharper swerve that is visibly less smooth than the APF paths.

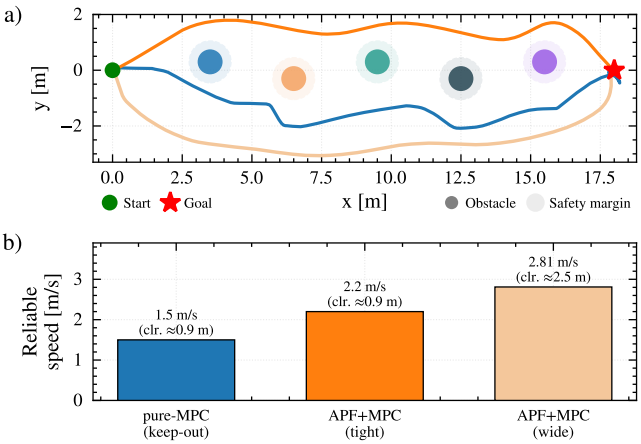


Fig. 2: Avoidance comparison (colours match across panels). (a) Paths for the three configurations; filled discs are obstacles, light rings the safety margin. (b) Maximum reliable speed (3/3 runs).

*Maximum reliable speed.* The commanded speed was swept and the highest value that passed reliably (3/3 runs) recorded; Fig. 2(b) summarises the result. Pure-MPC is reliable up to 1.5 m/s, APF+MPC (tight) up to 2.2 m/s, and APF+MPC (wide) up to 2.81 m/s. Although the wide planner keeps far more clearance, it is the *fastest*: its larger look-ahead lets it begin avoiding early and turn gently. Tightening the APF to the same clearance as pure-MPC removes this advantage and drops the maximum reliable speed to 2.2 m/s, only modestly above pure-MPC. The APF speed advantage therefore comes from its wider look-ahead, not from the architecture: at matched clearance the two are close.

*Shared failure mode.* Above this speed, every mode fails the same way (Fig. 3). As the lateral avoidance maneuver demands a larger tilt, the body thrust axis points increasingly sideways; once the attitude passes roughly 90° ($q_w < 0.7$) the thrust accelerates the vehicle horizontally instead of supporting it, producing a positive-feedback *attitude runaway*

(speed spikes from 2 to 9 m/s and altitude collapses) that the 1 s horizon cannot recover. This dynamic limit, not the solver (which stays at 8.6 ms), sets the maximum reliable speed.

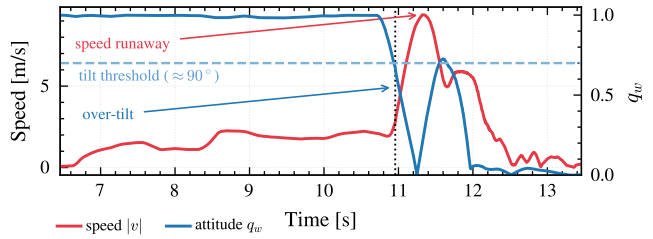


Fig. 3: Attitude runaway in a failed high-speed run. As $q_w$ drops past the ≈90° tilt threshold, the speed runs away and the vehicle crashes; the post-crash transient is cropped.

*Scope and outlook.* These results come from a single in-line slalom in which every obstacle leaves a free corridor on either side, so the wide planner can trade space for speed. The advantage it gains from extra clearance is therefore tied to having room to keep it: in a denser or more cluttered scene the obstacles would not permit a ≈ 2.5 m margin, and the wide and tight configurations would converge. A fuller picture requires sweeping obstacle density, corridor width and layout, which is left to future work; the present study isolates *where* avoidance is computed at a single matched clearance rather than mapping the full envelope.

Key metrics are summarised in Table II.

TABLE II: Obstacle-avoidance comparison (in-line slalom)

| Mode | Avoidance by | Min. clr. | Reliable speed |
|---|---|---|---|
| pure-MPC | MPC keep-out | ≈0.9 m | 1.5 m/s |
| APF+MPC (tight) | APF planner | ≈0.9 m | 2.2 m/s |
| APF+MPC (wide) | APF planner | ≈2.5 m | 2.81 m/s |

## B. Backflip

The backflip maneuver was executed from a 10 m hover altitude.

Fig. 4 shows the flight analysis with phase-shaded bands marking each of the five stages. In panel (a), the attitude error reaches 180° at the apex of the flip, while the pitch rate peaks at approximately 9.5 rad/s during the coast phase before the decelerating torque arrests the rotation; the full rotation completes in 0.73 s. Panel (b) shows the lateral drift from the hover position, which peaks at 1.7 m at flip exit; the NMPC then arrests the lateral motion and returns the vehicle to within 0.1 m of the hover point. Panel (c) tracks the altitude, which rises to approximately 12.0 m during the popup and flip and recovers to a stable hover without dropping below the initial altitude, since the fast rotation leaves little time to fall. Panel (d) shows the total thrust: it spikes to ≈ 25 N ($2mg$) during the popup, drops to near zero during coast, and oscillates as the NMPC regains control in recovery. During the acceleration and deceleration phases the feedforward flip torque $\tau_{\text{flip}}$ (Table I) is applied open-loop, exploiting the full actuator envelope beyond the NMPC

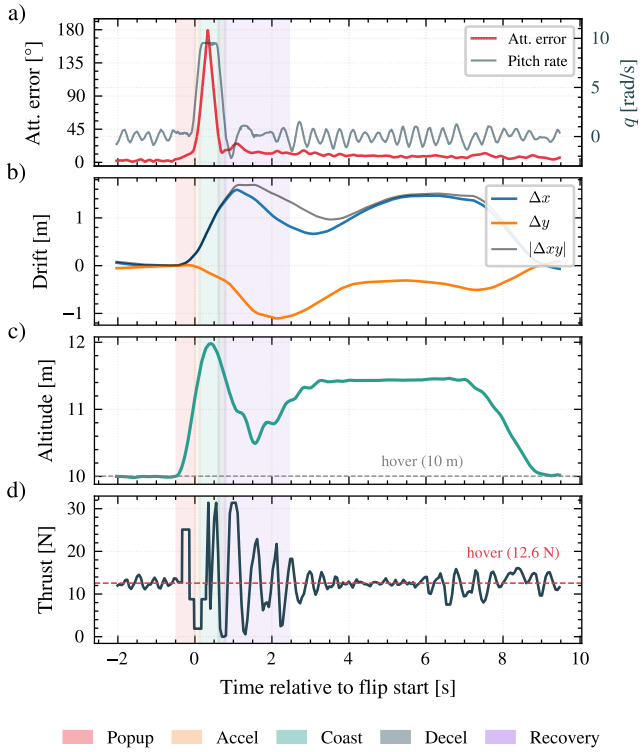


Fig. 4: Backflip flight analysis. (a) Attitude error and pitch rate. (b) Lateral drift from hover position. (c) Altitude. (d) Total thrust; the dashed line marks hover ($mg = 12.6$ N). Coloured bands indicate the five flip phases.

tracking bound. After the recovery band (purple in Fig. 4), the NMPC returns the vehicle to the hover point, where the drift and altitude settle back to their pre-flip values.

## C. MPC Solver Performance

For the receding-horizon controller to run online, every optimal control problem must be solved within the 50 ms sampling period so that the control loop keeps pace with the planned rate. The SQP-RTI solve time was therefore profiled across both flight scenarios.

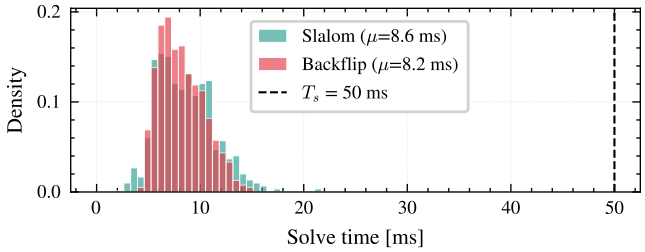


Fig. 5: NMPC solve time distribution for slalom and backflip scenarios. The dashed line marks the 50 ms sampling period $T_s$. All solves complete well below this budget.

The resulting distribution is shown in Fig. 5. The average solve time was 8.6 ms for the slalom and 8.2 ms for the backflip, with a maximum of 21.4 ms across all samples, comfortably within the 50 ms budget, which confirms the

real-time feasibility of the SQP-RTI approach. Table III summarises the solver statistics.

TABLE III: MPC Solve Time Statistics

| | Slalom | Backflip |
|---|---|---|
| Mean | 8.6 ms | 8.2 ms |
| Median | 8.3 ms | 7.9 ms |
| Max | 21.4 ms | 15.0 ms |
| $< 50$ ms | 100% | 100% |

## V. Conclusion

This study presented a quaternion NMPC framework and used it to compare two places to perform obstacle avoidance: as keep-out constraints *inside* the NMPC, or in an APF planner whose references the NMPC tracks. On an in-line slalom at a matched $\approx$0.9 m clearance, in-MPC avoidance was reliable to $1.5$ m/s and the APF+MPC tracker to $2.2$ m/s; the larger gap to the wide-clearance APF ($2.81$ m/s) is explained by look-ahead, not architecture, since the APF influence distance couples clearance and path smoothness. Both architectures share an attitude-runaway speed limit set by the vehicle dynamics rather than the solver (8.6 ms average, well within the 50 ms budget). Used alone as a controller, an APF has no such dynamic guarantees [1], [2]; the APF-reference plus NMPC-tracker hybrid combines smooth, look-ahead planning with the NMPC's dynamic feasibility and is the most capable configuration tested. Finally, an aggressive backflip drove the flip itself with a feedforward Lupashin profile [5], with the NMPC used only for recovery: starting from the post-flip excursion it left behind ($1.7$ m peak lateral drift), the same controller stabilised the vehicle back to within $0.1$ m of the hover point, showing it can recover from the violent attitude and position errors such a maneuver produces. The quaternion state representation kept attitude tracking singularity-free throughout. These findings come from a single corridor in which every obstacle leaves free space on either side; sweeping obstacle density, width and layout to map the full operating envelope, where the wide planner's clearance advantage is expected to shrink, remains future work.

## References


[1] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Int. J. Robot. Res.*, vol. 5, no. 1, pp. 90–98, 1986.

[2] S. S. Ge and Y. J. Cui, "New potential functions for mobile robot path planning," *IEEE Trans. Robot. Autom.*, vol. 16, no. 5, pp. 615–620, 2000.

[3] R. Verschueren, G. Frison, D. Kouzoupis *et al.*, "acados—a modular open-source framework for fast embedded optimal control," *Math. Program. Comput.*, vol. 14, no. 1, pp. 147–183, 2022.

[4] G. Frison and M. Diehl, "HPIPM: a high-performance interior-point method for quadratic programming and model predictive control," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 6563–6569, 2020.

[5] S. Lupashin, A. Schöllig, M. Sherback, and R. D'Andrea, "A simple learning strategy for high-speed quadrocopter multi-flips," in *Proc. IEEE ICRA*, Anchorage, AK, 2010, pp. 1642–1648.

[6] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *Proc. Eur. Control Conf. (ECC)*, 2019, pp. 3420–3431.

[7] S. Sun, A. Romero, P. Foehn, E. Kaufmann, and D. Scaramuzza, "A comparative study of nonlinear MPC and differential-flatness-based control for quadrotor agile flight," *IEEE Trans. Robot.*, vol. 38, no. 6, pp. 3357–3373, 2022.

[8] E. Kaufmann, A. Loquercio, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Deep drone acrobatics," in *Proc. Robotics: Science and Systems (RSS)*, 2020.